

Project Proposal

1. Clothes Classification

2. **Brief on the project:** This project classify the clothes. There are around 20 categories of clothes. Algorithm should classify the type of cloth.

3. **Deliverables of the project:** As part to solve this project we will be performing following activities.

- Import data into python env using panda.
- Load images from the hard disk and their labels from the csv file.
- Scale images to 224 X 224 size, which is best suited for VGG16 image classification model.
- Perform data augmentation for minority classes.

- List of questions your model/problem are designed to answer.

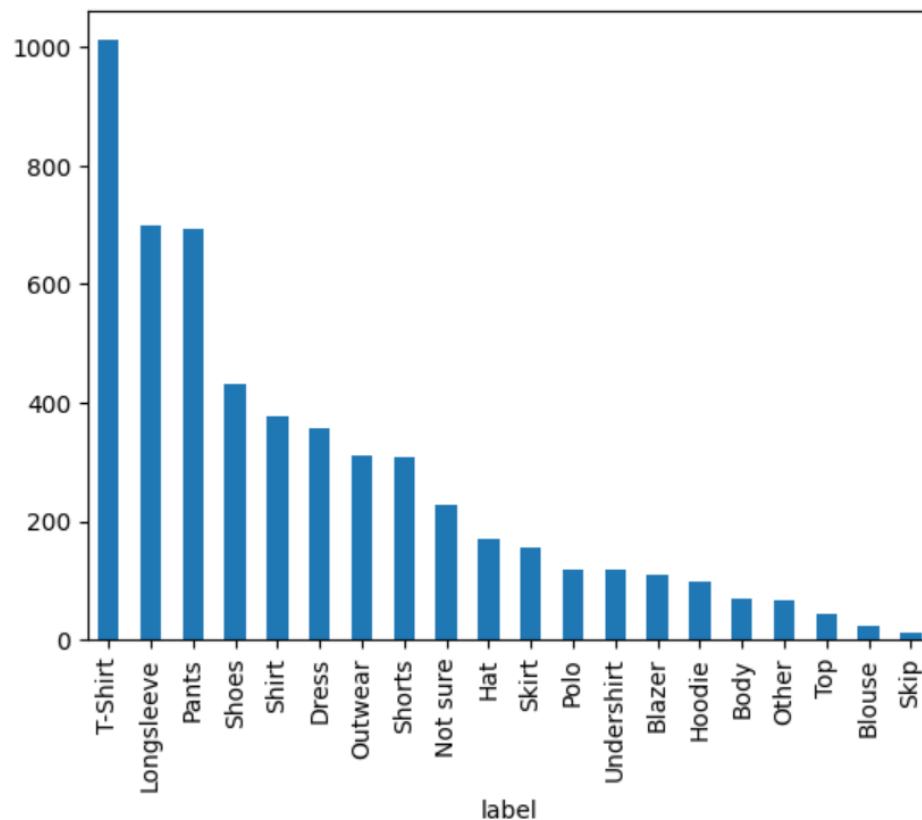
This model typically answers one question, what is the class of the given clothe. For this model is trained on approx 5000 records.

- Details of the model , important findings, expecting observations and outcome.

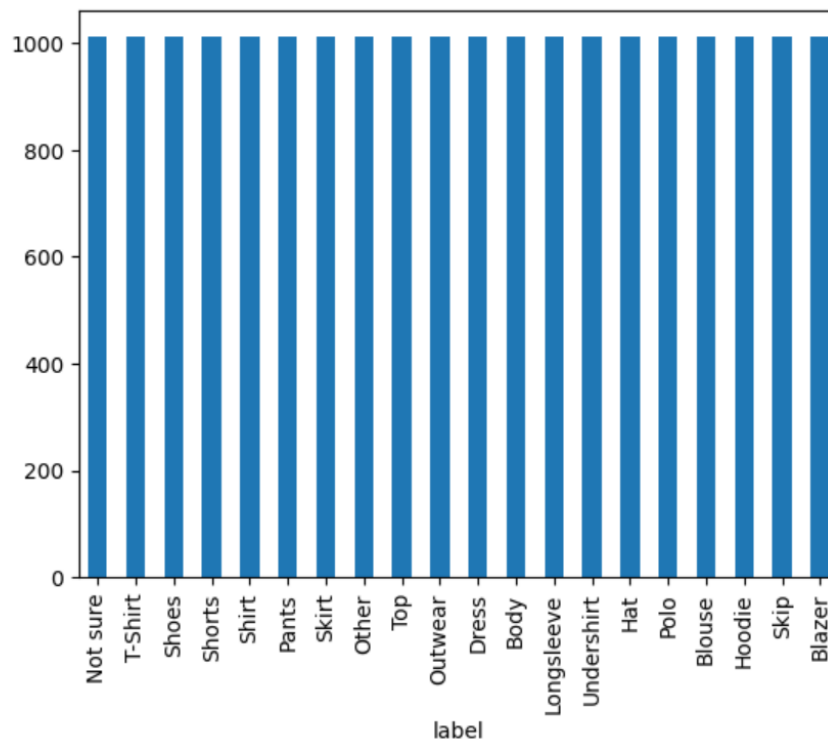
Evaluate different models and finalize the best model based on accuracy, precision and recall values.

We evaluated mainly three models and found VGG16 to be the best in terms of accuracy, precision and recall values.

- We observed variation in distribution of different classes. There is huge imbalance we tried to address it using data augmentation.



- Class distribution after applying data augmentation



4. Resources

- **Data set source:** We have used dataset provided at Kaggle website.
<https://www.kaggle.com/datasets/agrigorev/clothing-dataset-full/data/>
- **Software:** We have used variety of softwares and packages for this project:
 - a) Python
 - b) Panda
 - c) Sci-kit learn
 - d) Numpy
 - e) Visual studio code
 - f) Matplotlib
 - g) VGG16 pretrained model
 - h) Resnet50 pretrained model
 - i) Tensorflow
 - j) Simple CNN Model
 - k) Google colab

5. Individual Details:

Name: Wasim Bari

Mobile: 9650307130

Email: wbari31@gmail.com

6. Milestones

Milestones
Define a problem
Understanding the business problem
Get the Data
Explore and pre-process data
Choosing the python platform
Create Features
EDA
Create Model
Model Evaluation
Report Writing
Project submission

Define a problem:

There is a set of clothes given to the user. User can use the model to identify type of clothe. This would help user in pricing, sales prediction, inventory management.

Understanding the business problem:

This application can be applied in various business scenarios. One use case can be to identify attire of a person at a particular moment. Another could be inventory management for different clothe types by a cloth seller.

Getting data:

Data is available at Kaggle website.

EDA:

As part of EDA we performed following activities:

- Loading data into panda.

```
df_image=pd.read_csv('images.csv')
# Assigning numerical value to the label, since label is string and CNN
model works only with numerical data
label_indx={label:indx for indx,label in
enumerate(df_image['label'].unique())}
```

```
df_image['label_idx']=df_image['label'].map(label_idx)
```

- **Doing train and test split.**

```
# Train and test split
from sklearn.model_selection import train_test_split
train_df, test_df = train_test_split(df_image, test_size=0.2,
stratify=df_image["label_idx"],
random_state=7)
```

- **Loading images from the disk and associate them with corresponding label**

```
def load_image(path, label):
    image = tensorflow.io.read_file(path)
    image = tensorflow.image.decode_jpeg(image, channels=3)
    image = tensorflow.image.resize(image, (224, 224))
    image = image / 255.0    # normalize
    return image, label

# Converts dataframe into tensorflow dataset. Also club images into
batch of 32 images in each batch.
def create_tf_dataset(df):
    dataset =
tensorflow.data.Dataset.from_tensor_slices((df['image_path'],
df['label_idx']))
    dataset = dataset.map(load_image)
    dataset = dataset.batch(32)
    return dataset

train_df = create_tf_dataset(train_df)
test_df = create_tf_dataset(test_df)
```

- **Plotting graph for number of items belonging to different classes in the dataset**

```
df_image['label'].value_counts().plot(kind='bar')
```

- **Displaying 2 images from each category for reference.**

```
# Display two images from each class or category
def display_image(image, label):
    plt.imshow(image.numpy())
    plt.axis("off")
    plt.title(f"Label: {label_dict[label.numpy()]}")
    plt.show()

img_count_dict={key:0 for key in label_dict.keys()}
print(img_count_dict)
```

```
# Displaying 2 images of each category
ds = train_df.unbatch()
cntr=0

for image, label in ds:
    key = int(label.numpy())
    if img_count_dict.get(key)<2 :
        img_count_dict[key] = img_count_dict.get(key) + 1
        cntr = cntr+1
        display_image(image, label)

    if all(value > 2 for value in img_count_dict.values()):
        break

display(cntr)
print(img_count_dict)
```

- Creating simple CNN model

```
# Creating simple CNN model.
# Simple CNN model
# CNN Model
simple_cnn_model = Sequential()
simple_cnn_model.add(Input(shape=(224,224,3), name='input_layer'))

simple_cnn_model.add(Conv2D(128,kernel_size=(3,3),padding='valid',activation='relu', name='conv_1'))
simple_cnn_model.add(BatchNormalization())
simple_cnn_model.add(MaxPooling2D(pool_size=(2,2), strides=2,
padding='valid',name='maxpool_1'))

simple_cnn_model.add(Conv2D(256,kernel_size=(3,3),
padding='valid',activation='relu', name='conv_2'))
simple_cnn_model.add(BatchNormalization())
simple_cnn_model.add(MaxPooling2D(pool_size=(2,2), strides=2,
padding='valid',name='maxpool_2'))

# simple_cnn_model.add(Conv2D(256,kernel_size=(3,3),
padding='valid',activation='relu', name='conv_3'))
# simple_cnn_model.add(BatchNormalization())
# simple_cnn_model.add(MaxPooling2D(pool_size=(2,2), strides=2,
padding='valid',name='maxpool_3'))

simple_cnn_model.add(Flatten())
```

```
simple_cnn_model.add(Dense(256, activation='relu',
name='ann_hidden_1'))
#simple_cnn_model.add(Dropout(.1))

#simple_cnn_model.add(Dense(128, activation='relu',
name='ann_hidden_2'))
#simple_cnn_model.add(Dropout(.1))

simple_cnn_model.add(Dense(20, activation='softmax',
name='output_layer'))

print(simple_cnn_model.summary())
simple_cnn_model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy',metrics=['accuracy'])
# Early stopping for all models
early_stop = EarlyStopping(monitor='val_loss', patience=5,
restore_best_weights=True, verbose=1)
class_weights = class_ratios
```

- **Training simple CNN model**

```
simple_cnn_model_history=simple_cnn_model.fit(train_df, epochs=50,
validation_data=test_df, verbose=1, callbacks=[early_stop])
```

- **Drawing accuracy and loss graphs:**

```
draw_accuracy_plot(simple_cnn_model_history)
draw_val_plot(simple_cnn_model_history)
```

- **Printing test and training accuracy stats**

```
print_metrics(test_df, simple_cnn_model,'test')
print_metrics(train_df, simple_cnn_model,'train')
```

- **Using VGG16 model for transfer learning**

```
# Transfer learning using VGG16
from tensorflow.keras.applications import VGG16
def get_vgg16_transferlearning_model():
    vgg16_base = VGG16(weights='imagenet', include_top=False,
input_shape=(224,224,3))
    vgg16_base.trainable = False

    model = Sequential()
    model.add(vgg16_base)
    model.add(Flatten())
    model.add(Dense(128, activation='relu', name='ann_hidden_1'))
    model.add(Dropout(.1))
    model.add(Dense(20, activation='softmax', name='output_layer'))
```

```
return model
```

```
vgg16_transferlearning_model = get_vgg16_transferlearning_model()  
vgg16_transferlearning_model.summary()
```

- **Training VGG16 model on clothes dataset:**

```
vgg16_transferlearning_model.compile(optimizer='adam',  
loss='sparse_categorical_crossentropy',  
metrics=['accuracy'])  
vgg16_transferlearning_model_history=vgg16_transferlearning_model.fit(t  
rain_df, epochs=30, validation_data=test_df  
  
verbose=1, callbacks=[early_stop])
```

- **Using resnet50 model for transfer learning**

```
# Transfer learning using ResNet50  
from tensorflow.keras.applications import ResNet50  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D  
from tensorflow.keras.optimizers import Adam  
  
def get_resnet50_transferlearning_model():  
    resnet50_base = ResNet50(  
        weights='imagenet',  
        include_top=False,  
        input_shape=(224, 224, 3)  
    )  
  
    # Freeze base model  
    resnet50_base.trainable = False  
  
    # Build transfer learning model  
    model = Sequential()  
    model.add(resnet50_base)  
    model.add(GlobalAveragePooling2D())  
    model.add(Dense(128, activation='relu', name='ann_hidden_1'))  
    model.add(Dense(20, activation='softmax')) # 20 classes  
  
    # Compile  
    optimizer_adam = Adam(learning_rate=1e-4)  
    model.compile(  
        optimizer=optimizer_adam,  
        loss='sparse_categorical_crossentropy',  
        metrics=['accuracy']
```

```
)

return model

resnet50_transferlearning_model = get_resnet50_transferlearning_model()
resnet50_transferlearning_model.summary()

- Training resnet50 model on clothes dataset
resnet50_transferlearning_model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
resnet50_transferlearning_model_history=resnet50_transferlearning_model
.fit(train_df, epochs=50,
validation_data=test_df, verbose=1, callbacks=[early_stop])

- Performing data augmentation
df_image_2=df_image.copy()
new_rows = [] # This list keeps track of all newly created images

datagen = ImageDataGenerator(
    rotation_range=30,
    width_shift_range=0.2,
    height_shift_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

# Method to create images
def augment_and_save(img, datagen, save_path):
    aug_iter = datagen.flow(img, batch_size=1)
    aug_img = next(aug_iter)[0].astype('uint8')
    cv2.imwrite(save_path, aug_img)

for cls in label_short_fall_count.keys():

    required_images = label_short_fall_count[cls] # Return how many
images for a particular class we need to create.
    if required_images == 0:
        print(f"Class {cls} is already balanced. Skipping
augmentation.")
        continue

    images_df = df_image_2[df_image_2['label_indx'] == cls] # Select
all images already present for particular class only.
```

```
count = 0 # keeps track number of images created so far. Ideally
required_images + count = number of images of majority class.

print(f"Augmenting class {cls} with {required_images} images.")
while count < required_images: # iterate as many times as number of
images for a particular class to be created.
    single_image_df=images_df.sample(n=1) # Select one image of a
particular class randomly, from available images.
    img=single_image_df['image_path'].values[0]
    img = cv2.imread(img)

    if img is None:
        continue

    img = np.expand_dims(img, axis=0)
    new_name =
f"aug_{single_image_df['image'].values[0].split('.')[0]}_{count}" # All
generated image starts with 'aug' literal
    save_path = os.path.join("images_compressed_2",
new_name+".jpg")
    augment_and_save(img, datagen, save_path)

    new_rows.append({
        'image': new_name,
        'image_path': save_path,
        'label_indx': cls,
        'label': label_dict[cls]
    })
    count += 1

    # temp condition for testing remove it.
    # if count>0:
    #     break

print(f"Finished augmenting class {cls}. Total augmented images:
{count}")
```

Model Evaluation:

We used testing against following models:

- Custom Simple CNN Model
- VGG16
- Resnet50

- VGG16 with data augmentation
- Resnet50 with data augmentation

Given below is the table for model comparison:

S. No	Model	Accuracy		Precision		F1 Score	
		Training	Testing	Training	Testing	Training	Testing
1	Custom CNN Model	27%	26%	52%	16%	17%	16%
2	VGG16	83%	72%	84%	72%	81%	69%
3	Resnet50	21%	20%	14%	10%	10%	9%
4	VGG16 with Data augmentation	89%	67%	87%	68%	87%	66%
5	resent50 with Data augmentation	19%	18%	19%	17%	15%	15%

Final decision: VGG16 pretrained model gives best values for Accuracy, Precision and F1 Score for testing and training data. VGG16 with data augmentation shows overfitting problem.